

Room Matters: Dynamic Room-level Collaboration Information Modeling for Live Streaming Recommendation

Ke Guo Gaoling School of Artificial Intelligence Renmin University of China Beijing, China imgkkk2004@ruc.edu.cn	Changle Qu Gaoling School of Artificial Intelligence Renmin University of China Beijing, China changlequ@ruc.edu.cn	Xiao Zhang* Gaoling School of Artificial Intelligence Renmin University of China Beijing, China zhangx89@ruc.edu.cn
Liqin Zhao Shijun Wang Kuaishou Technology Beijing, China {zhaoliqin,wangshijun03}@kuaishou.com	Yanan Niu Kuaishou Technology Beijing, China niuyan@kuaishou.com	Jun Xu Gaoling School of Artificial Intelligence Renmin University of China Beijing, China junxu@ruc.edu.cn

Abstract

Live streaming platforms have recently gained popularity due to their immediacy and entertainment value, highlighting the need for streaming recommender systems that can adapt to the dynamic nature of evolving content, real-time interactions, and changing user interests. The “live room” plays a central role in modeling this dynamic environment, as it not only connects users with streamers but also serves as a key channel for collecting fine-grained user feedback. More specifically, the frequent interactions of users within a live room provide detailed dynamic collaboration information, reflecting the streamer’s real-time topic and users’ dynamic interests. However, existing studies have not thoroughly investigated the dynamics of room-level collaborative information. In this paper, we address this gap by emphasizing two perspectives: *the evolving tripartite interaction information among rooms, streamers, and users*, and *the real-time intra-room collaboration information*. We propose **DCGLive**, a **D**ynamic **C**ollaboration-aware **G**raph learning approach for **L**ive streaming recommendation. Specifically, we first construct two dynamic bipartite graphs to perceive the evolving tripartite interaction and generate real-time representations of streamers, rooms, and users. To account for the dynamic nature of live streaming, we design a set of non-parametric, collaboration-aware indicators that weight intra-room interactions based on both temporal recency and frequency, while guiding the embedding updating process for both users and rooms. Additionally, to address the cold-start challenge of newly created live rooms in real time, we propose a room representation initialization mechanism that balances both the relevance and the dynamics among different rooms hosted by the same streamer. Experiments conducted on both commercial and public datasets demonstrate that DCGLive consistently outperforms the baseline models. Our code is available at <https://github.com/imgkkk574/DCGLive>.

*Corresponding author: Xiao Zhang (e-mail: zhangx89@ruc.edu.cn).



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792241>

CCS Concepts

• Information systems → Recommender systems.

Keywords

Live Streaming Recommendation, Dynamic Graph, Intra-room Collaboration

ACM Reference Format:

Ke Guo, Changle Qu, Xiao Zhang, Liqin Zhao, Shijun Wang, Yanan Niu, and Jun Xu. 2026. Room Matters: Dynamic Room-level Collaboration Information Modeling for Live Streaming Recommendation. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792241>

1 Introduction

With the rapid development of mobile internet technology, live streaming has rapidly risen to prominence in recent years [5], profoundly reshaping how people consume and interact with online content. A defining characteristic of live streaming is the central role of the live room, where streamers broadcast and interact with users in real time, and where users engage through diverse activities such as liking and gifting. Unlike traditional media such as movies, books, or pre-recorded videos, live streaming is inherently dynamic and real-time [1, 10, 20, 26]. Content is simultaneously produced and consumed within a room, enabling immediate and reciprocal interaction between streamers and users. This dynamic setting causes user interests to fluctuate rapidly across different live rooms, often reflected in behavioral signals such as varying watch durations. Consequently, recommender systems for live streaming must address the highly dynamic evolution of user preferences to more effectively model and predict engagement.

Existing efforts to capture the dynamics of live streaming primarily focus on data timeliness and content modeling. For instance, Sliver [20] optimizes label accuracy via a 30-second sliding window, while ContentCTR [8] and MMBee [10] leverage multi-modal features to predict stream highlights. LARM [21] utilizes multi-modal large language models (LLMs) to interpret the real-time semantics of live content and align them with user preferences.

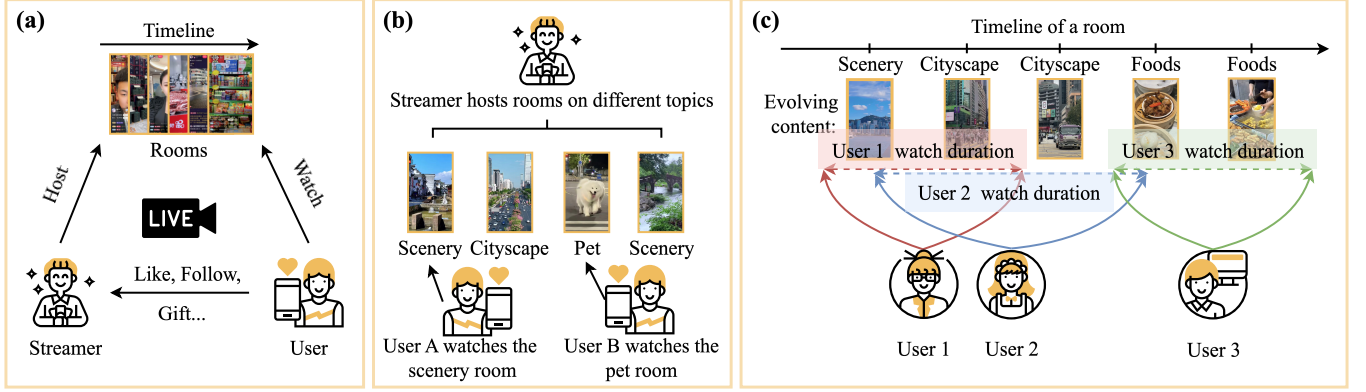


Figure 1: The dynamics of room-level collaboration information. (a): Example of the live streaming scenario with the tripartite interactions among streamers, rooms, and users. (b): As the streamer can cover various topics in different rooms, neglecting the interaction between users and rooms may result in missing the users' detailed interests. (c): Even within a room, its content changes over time. Some users' interests may not overlap, even though they are watching the same room. This highlights the importance of considering real-time intra-room collaboration information.

However, the dynamics of room-level collaborative information have not been thoroughly investigated in prior research. These dynamics manifest in two complementary aspects: (1) **The evolving tripartite interaction among streamers, rooms, and users.** As illustrated in Figure 1(a), streamers host live rooms on different topics at various times, while users frequently switch between rooms and engage with multiple streamers. Such interactions reflect distinct facets of user interests: long-term preferences are often captured by enduring behaviors such as gifting from users to streamers, whereas short-term, real-time preferences are revealed through immediate interactions with rooms, such as liking and chatting. Moreover, as shown in Figure 1(b), users may exhibit different interests when consuming content from the same streamer but across different rooms, since each room represents unique contextualized content. This highlights the necessity of modeling the tripartite relationship at the room level to preserve fine-grained collaboration signals. (2) **The real-time intra-room collaboration information.** As depicted in Figure 1(c), even within the same live room, the content evolves continuously, leading to shifts in user interests over time. Users whose watch durations substantially overlap are likely to share similar real-time preferences, while those with little overlap may diverge in their interests. Neglecting such intra-room collaboration overlooks critical signals that are essential for representing user dynamics and the evolving semantics of live rooms. Overall, the “live room” serves as the pivotal unit for capturing the dynamics mentioned above, as it acts as a bridge between users and streamers while providing rich and fine-grained feedback signals. By explicitly incorporating room-level collaborative information, recommender systems can better capture subtle shifts in user preferences and the characteristics of real-time live content.

In this paper, we propose **DCGLive**, a **D**ynamic **C**ollaboration-aware **G**raph learning approach for **L**ive streaming recommendation, which leverages room-level collaboration information to better capture dynamic evolving user preferences and enhance recommendation performance. Specifically, DCGLive captures the evolving representations of streamers, rooms, and users by constructing two

dynamic bipartite graphs: user-room and user-streamer. These representations are updated with each new interaction through a third-order updating mechanism, effectively capturing both collaborative and sequential relationships inherent in real-time interactions. To address the cold-start issue faced by newly created live rooms, we propose a room representation initialization mechanism that effectively incorporates both relevance and dynamics across various rooms hosted by the same streamer. This mechanism implicitly leverages the relationships between streamers and rooms, thereby enhancing the representation learning for rooms within a limited timeframe. Furthermore, to evaluate and leverage the impact of intra-room collaboration information on representation learning, we develop the Live Collaboration-aware Indicator (LiveCI). This non-parametric module assesses the effectiveness of collaboration within a room based on both temporal recency and interaction frequency, subsequently guiding the representation updating for both users and rooms. In this manner, DCGLive can effectively capture fine-grained, dynamic intra-room collaborative signals.

To verify the effectiveness of DCGLive, we conduct experiments on the next room prediction task and compare it with nine baseline methods. In summary, the contributions of this work are as follows:

- We identify the dynamics of room-level collaboration information in live streaming from two perspectives: *the evolving tripartite relationships among streamers, rooms, and users*, and *the real-time intra-room collaboration information*.
- We propose DCGLive, a dynamic collaboration-aware dual-graph learning approach that incorporates a room initialization mechanism and LiveCI to capture both evolving tripartite relationships and fine-grained intra-room collaborative information.
- Experimental results demonstrate that DCGLive outperforms baseline methods on both commercial and public datasets.

2 Related Work

Live Streaming Recommendation. Early studies focus on modeling the relationship between users and streamers [3, 26]. For example, the pioneering work LiveRec [26] employs a self-attention

mechanism to capture repeated consumption patterns between users and streamers. Another line of research incorporates short video data into live streaming recommendation to alleviate data sparsity [1, 18, 25]. More recently, researchers have emphasized the dynamic nature of live streaming. Several studies approach dynamics from the perspective of live content [8–10, 21, 22, 49]. From a collaborative perspective, DRIVER [11] introduces a dynamic learning approach that models inter-room relationships based on users' behavioral paths after leaving a room. In addition, LSEC-GNN [43] explores the tripartite interaction among users, streamers, and products in live streaming e-commerce. Despite these advancements, the dynamic nature of room-level collaborative information remains largely overlooked, presenting a significant research gap that requires further exploration.

Dynamic Modeling in Recommendation. In streaming recommendation, capturing dynamics is crucial as user interests evolve over time [2, 28, 45, 46]. Some efforts have been made to address this challenge [4, 7, 29, 33, 35, 47]. To incorporate temporal dynamics into graph-based collaborative filtering, dynamic graph neural networks (DGNNs) have been proposed [13, 17, 34, 38, 39, 41, 42, 44, 48, 52]. DGNNs can be categorized into discrete-time dynamic graphs (DTDGs) and continuous-time dynamic graphs (CTDGs) based on time granularity. DTDGs, such as EvolveGCN [23], update GNN parameters across snapshots but face challenges in selecting appropriate intervals and capturing intra-span dynamics. In contrast, CTDGs model temporal events continuously, enabling finer-grained dynamic representation. Several DGNN-based methods have been applied to recommendation [19, 30, 32, 50, 51]. DGCF [19] introduces embedding updating mechanisms to capture collaborative and sequential relations, while DGEL [32] enhances these mechanisms with re-scaling networks for consistent dynamic embeddings. Despite DGNNs' suitability for dynamic scenarios, their application in live streaming recommendation remains unexplored. We address this gap by constructing bipartite CTDGs to model tripartite relationships and maintaining real-time representations.

3 Problem Formulation

Formally, a CTDG $\mathcal{G}$ evolves sequentially through a series of graph events, where each event represents possible modifications to the graph, such as node/edge additions or deletions, or updates to node features. These modifications result in changes to the structure or attributes of the graph. In the context of live streaming recommendation, we consider user watching behavior in live rooms and their gifting behavior towards streamers, focusing exclusively on edge addition events. Let $\mathcal{S}$, $\mathcal{R}$, and $\mathcal{U}$ denote the sets of streamers, live rooms, and users, respectively. An interaction $I_t = (u_t, r_t, s_t, w_t, g_t, t)$ represents a user engagement event at timestamp $t \in \mathcal{T}$, where user $u_t \in \mathcal{U}$ enters live room $r_t \in \mathcal{R}$ hosted by streamer $s_t \in \mathcal{S}$. w_t represents the watch duration, while g_t denotes the number of gifts that user u_t sends to streamer s_t during this interaction. If no gifts are sent, then $g_t = 0$. Here, $\mathcal{T}$ denotes the set of timestamps belonging to $\mathbb{R}^+$. Based on this formulation, we define two CTDGs: user-room and user-streamer graphs.

Definition 1. User-Room (U-R) Graph. The user-room graph is denoted as $\mathcal{G}^{\text{UR}} = (\mathcal{U} \cup \mathcal{R}, \mathcal{E}^{\text{UR}})$, where $\mathcal{U} \cup \mathcal{R}$ represents the

node set and $\mathcal{E}^{\text{UR}}$ denotes the edge set constructed based on watching behaviors between users and rooms. The initial graph $\mathcal{G}_{t_0}^{\text{UR}}$ at timestamp t_0 consists of isolated nodes with no edges. For each interaction at timestamp t , the corresponding event in $\mathcal{G}^{\text{UR}}$ is represented as (u_t, r_t, w_t, t) , indicating that a new edge is established between nodes u_t and r_t with edge feature w_t .

Definition 2. User-Streamer (U-S) Graph. The user-streamer graph is denoted as $\mathcal{G}^{\text{US}} = (\mathcal{U} \cup \mathcal{S}, \mathcal{E}^{\text{US}})$, where $\mathcal{U} \cup \mathcal{S}$ represents the node set and $\mathcal{E}^{\text{US}}$ denotes the edge set constructed based on gifting behaviors between users and streamers. The initial graph $\mathcal{G}_{t_0}^{\text{US}}$ at timestamp t_0 consists of isolated nodes with no edges. For each interaction at timestamp t , if $g_t \neq 0$, the corresponding event in $\mathcal{G}^{\text{US}}$ is represented as (u_t, s_t, g_t, t) , indicating that a new edge is established between nodes u_t and s_t with edge feature g_t .

Given the dynamic graphs constructed from interactions among streamers, rooms, and users, the objective of **live room recommendation** is to learn and update tripartite representations based on historical interactions and current events, and then leverage these learned representations to predict the live room with which a user is most likely to interact at the next timestamp. Formally, this objective can be expressed as follows:

$$\mathcal{G}_t^{\text{UR}}, \mathcal{G}_t^{\text{US}} = \mathcal{P}(\mathcal{G}_{t^-}^{\text{UR}}, \mathcal{G}_{t^-}^{\text{US}}, I_t), \quad r_{\text{predict}, t^+} = \mathcal{F}(\mathcal{G}_t^{\text{UR}}, \mathcal{G}_t^{\text{US}}),$$

where t^- and t^+ denote the nearest preceding and succeeding timestamps relative to timestamp t , respectively. $\mathcal{P}$ represents the graph updating mechanism, while $\mathcal{F}$ denotes the prediction function.

4 The Proposed Approach: DCGLive

We propose DCGLive (Dynamic Collaboration-aware Graph learning approach for Live streaming recommendation), which captures both evolving tripartite relationship and real-time intra-room collaboration information to enhance live streaming recommendation.

4.1 Overview of DCGLive

The overall idea of DCGLive is to model dynamic room-level collaboration in a structured and adaptive manner. As shown in Figure 2, DCGLive operates in three key steps. First, it constructs dynamic user-room and user-streamer graphs to capture the evolving tripartite interactions. Second, to mitigate the cold-start issue of newly created rooms, a room initialization mechanism is designed by incorporating both relevance and dynamics across various rooms hosted by the same streamer. Finally, a set of non-parametric collaboration-aware indicators (LiveCI) is introduced to weight the fine-grained intra-room collaborative effects and then guide real-time representation updating for both users and rooms. Through this workflow, DCGLive maintains real-time and collaboration-sensitive representations that are used for accurate next-room prediction.

4.2 Dynamic User-Room & User-Streamer Graph Construction

As discussed in Section 1, streamers can host different live rooms on various topics, and users can frequently switch between rooms and interact with different streamers. This evolving tripartite interaction among streamers, rooms, and users contains crucial collaboration information. To perceive this relationship, we first construct two

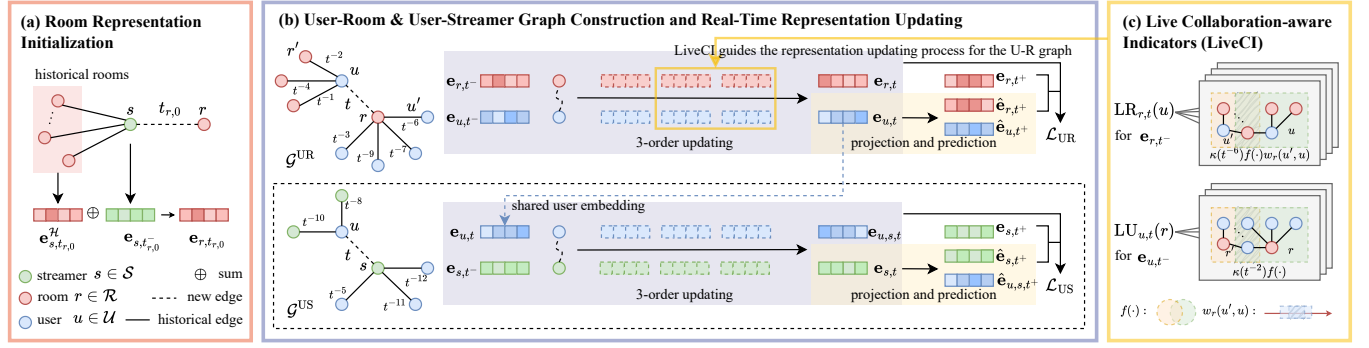


Figure 2: Overall framework of DCGLive, where e_{r,t^-} , e_{u,t^-} , e_{s,t^-} denote the representations of room r , user u , and streamer s at the previous timestamp t^- , respectively, and $e_{r,t}$, $e_{u,t}$, $e_{s,t}$ are the updated representations at the current timestamp t .

dynamic graphs based on the Definitions 1, 2: the user-room (U-R) graph $\mathcal{G}^{UR}$ and the user-streamer (U-S) graph $\mathcal{G}^{US}$. The U-R graph effectively captures the frequent watching behaviors between users and rooms, while the U-S graph leverages the gifting behavior of users towards streamers. When new interactions occur, the structures and node representations of the corresponding graph are updated accordingly. The sequence information of live rooms previously hosted by the same streamer is implicitly utilized within the graphs, thereby establishing connections between the representations of streamers and rooms. Notably, the user representations in these two graphs are shared, allowing a user's representation to be updated through both watching behaviors with rooms and gifting to streamers, thereby capturing collaborative information from both the room and streamer perspectives.

4.3 Room Representation Initialization

When a streamer creates a new live room, the initial lack of interactions often leads to a cold-start problem, as also noted by Qu et al. [24]. Additionally, given that the live room is available for only a limited duration, it is essential to learn an effective representation of it within this timeframe. To tackle this issue, we design a room representation initialization mechanism that leverages specific features from the streamer side as the initial representation of the new room, rather than relying on the random initialization typically employed by general DGNNs [17, 19]. As shown in Figure 2(a), our mechanism integrates two complementary factors: the historical live rooms and the most recent room. The latest room is prioritized due to live streaming's dynamic nature. Streamers often create consecutive rooms on related topics, so emphasizing the most recent representation better reflects their current state while preserving long-term context. In this way, our approach balances long-term consistency with short-term dynamics, while implicitly leveraging the intrinsic relationships between streamers and rooms.

Formally, if streamer s creates a new room r at timestamp $t_{r,0}$, the initial embedding $e_{r,t_{r,0}}$ of room r can be computed as follows:

$$e_{s,t_{r,0}}^{\mathcal{H}} = \frac{1}{|\mathcal{H}_{s,t_{r,0}}|} \sum_{r_j \in \mathcal{H}_{s,t_{r,0}}} \bar{e}_{r_j}, \quad e_{r,t_{r,0}} = \frac{e_{s,t_{r,0}}^{\mathcal{H}} + e_{s,t_{r,0}}^-}{2},$$

where $e_{s,t_{r,0}}^{\mathcal{H}}$ represents the past room characteristics of streamer s , which combines the final embeddings of s 's historical rooms. Here,

$\mathcal{H}_{s,t_{r,0}}$ denotes the sequence of live rooms that s has hosted prior to timestamp $t_{r,0}$, and $\bar{e}_{r_j}$ represents the final embedding of room r_j . The representation $e_{r,t_{r,0}}$ is derived by combining $e_{s,t_{r,0}}^{\mathcal{H}}$ with s 's previous representation $e_{s,t_{r,0}}^-$ at the most recent prior timestamp $t_{r,0}^-$.

4.4 Live Collaboration-Aware Indicators (LiveCI)

Real-time collaborative relationships naturally emerge among users within the same live room. However, given the varying frequencies and durations of user-room interactions, it is unreasonable to assume that each interaction contributes equally to the overall collaborative effect. In particular, users with larger overlaps in watching duration and a higher number of co-watched rooms are more likely to share similar real-time interests. To effectively capture these differences, we introduce LiveCI, a non-parametric module inspired by [31, 37]. It is designed to evaluate the strength of intra-room collaboration and guide the representation updating process for the user-room graph, enabling the model to better leverage fine-grained collaboration information. As illustrated in Figure 2(c), it consists of two indicators, LiveCI for User (LU) and LiveCI for Room (LR), which respectively assess the temporal collaborative impact on users and rooms. Both indicators are sensitive to the recency of interactions, consistent with the temporal dynamics observed in [31], while LR additionally accounts for users' overlapping watch duration to more accurately measure the real-time alignment of their interests. They are formally computed as:

$$\begin{aligned} LU_{u,t}(r) &= \sum_{z \in \mathcal{H}_{u,t}} \kappa(z, \mathcal{H}_{u,t}) f(\{\mathcal{H}_{z,t} \cup z\}, \{\mathcal{H}_{r,t} \cup r\}), \\ LR_{r,t}(u) &= \sum_{x \in \mathcal{H}_{r,t}} \kappa(x, \mathcal{H}_{r,t}) f(\{\mathcal{H}_{x,t} \cup x\}, \{\mathcal{H}_{u,t} \cup u\}) w_r(x, u, \mathcal{H}_{x,t}), \end{aligned} \quad (1)$$

$$\begin{aligned} \kappa(z, \mathcal{H}_{u,t}) &= \exp\{-[t - t(z)]\} \Bigg/ \sum_{z' \in \mathcal{H}_{u,t}} \exp\{-[t - t(z')]\}, \\ \kappa(x, \mathcal{H}_{r,t}) &= \exp\{-[t - t(x)]\} \Bigg/ \sum_{x' \in \mathcal{H}_{r,t}} \exp\{-[t - t(x')]\}, \end{aligned} \quad (2)$$

where $\mathcal{H}_{u,t}$ and $\mathcal{H}_{x,t}$ represent the historical watching room sets of user u and x , respectively. Similarly, $\mathcal{H}_{r,t}$ and $\mathcal{H}_{z,t}$ denote the historical user sets of room r and z at the current timestamp t . $w_r(x, u, \mathcal{H}_{x,t}) = 1 + \text{owt}_r(x, u)$, which computes the overlapping

watching weight, and $\text{owt}_r(x, u)$ represents the normalized score of overlapping watch duration between user x and user u within room r . The function $\kappa(\cdot)$ returns the time-decay weight, while $t(x)$ and $t(z)$ represent the timestamps of historical interactions with neighbors x and z , respectively. The function $f(\cdot)$ computes the similarity between two sets. Following [31, 37], we define $f(\cdot)$ as the Jaccard similarity. The computation is illustrated in Figure 2 (c).

When a new interaction between user u and room r occurs at the current timestamp t , $\text{LU}_{u,t}(r)$ examines the intra-room collaborative impact from room r to the target user u , while $\text{LR}_{r,t}(u)$ represents this impact from user u to the target room r . As discussed in Section 1, the overlapping watch duration of two users within the same room can imply their differing interests, which in turn influences their collaboration information. Therefore, $\text{LR}_{r,t}(u)$ takes into account the overlapping watching weight $w_r(x, u, \mathcal{H}_{x,t})$ between user u and other viewers within room r . However, since it is not possible for user u to watch two live rooms simultaneously, the calculation of $\text{LU}_{u,t}(r)$ does not need to consider this factor. Ultimately, these two indicators will be applied to the representation learning process of the U-R graph (see Section 4.5.4).

4.5 Real-Time Representation Updating

To achieve real-time node representation, the structure of the CTDG and the node embedding must be updated whenever a new interaction occurs. Following DGCF [19], the embedding updating process involves three mechanisms: zero-order inheritance, first-order propagation, and second-order aggregation. Both the U-R graph and the U-S graph utilize these three mechanisms. Therefore, to simplify the expression of formulas, we use item i to denote either a room or a streamer within these mechanisms. When a new interaction between user u and item i occurs at timestamp t , the node embeddings of user u and item i will be updated according to the following three mechanisms and then fused to obtain the new embeddings.

4.5.1 Zero-Order Inheritance. In a dynamic graph, nodes awaiting updates are influenced primarily by their previous states. Additionally, the time interval between two interactions also impacts the learning of embeddings [17]. Therefore, the zero-order inheritance encoding of these factors can be computed as follows:

$$\begin{aligned} \mathbf{e}_{u,t}^{(0)} &= \phi_u \left(\mathbf{W}_0^u \mathbf{e}_{u,t^-} + \mathbf{w}_0 \Delta t_u + \mathbf{W}_0^f \mathbf{f}_u \right), \\ \mathbf{e}_{i,t}^{(0)} &= \phi_i \left(\mathbf{W}_0^i \mathbf{e}_{i,t^-} + \mathbf{w}_0 \Delta t_i + \mathbf{W}_0^f \mathbf{f}_i \right), \end{aligned} \quad (3)$$

where Δt_u and Δt_i denote the time intervals between the current timestamp t and the previous interaction timestamp t^- , $\mathbf{e}_{u,t^-}^{(0)}$ and $\mathbf{e}_{i,t^-}^{(0)}$ represent the user u and item i embeddings after the interaction at time t , derived from zero-order inheritance. Meanwhile, $\mathbf{e}_{u,t^-}$ and $\mathbf{e}_{i,t^-}$ are the most recent embeddings of user u and item i at time t^- . The vectors $\mathbf{f}_u$ and $\mathbf{f}_i$ correspond to the features of the user and the item, respectively. $\mathbf{W}_0^u, \mathbf{W}_0^i, \mathbf{W}_0^f \in \mathbb{R}^{d \times d}$ are learnable parameter matrices, and $\mathbf{w}_0 \in \mathbb{R}^d$ is the parameter vector used to encode the time interval Δt . ϕ_u and ϕ_i serve as activation functions.

4.5.2 First-order Propagation. In live streaming recommendation, the rooms that a user watches serve as direct reflections of the user's recent intentions and interests. Similarly, users who interact with a specific room collectively define the characteristics of

that room. The same principle applies to gifting behaviors between users and streamers. Thus, new interactions play a pivotal role in updating embeddings. Based on this intuition, first-order propagation leverages information from first-order neighbors to learn user and item representations, which can be computed as follows:

$$\mathbf{e}_{u,t}^{(1)} = \phi_u \left(\mathbf{W}_1^u \mathbf{e}_{i,t^-} + \mathbf{W}_1^f \mathbf{f}_i \right), \quad \mathbf{e}_{i,t}^{(1)} = \phi_i \left(\mathbf{W}_1^i \mathbf{e}_{u,t^-} + \mathbf{W}_1^f \mathbf{f}_u \right), \quad (4)$$

where $\mathbf{e}_{u,t}^{(1)}$ and $\mathbf{e}_{i,t}^{(1)}$ denote the embeddings of user u and item i , respectively, obtained from the first-order propagation. The matrices $\mathbf{W}_1^u, \mathbf{W}_1^i, \mathbf{W}_1^f \in \mathbb{R}^{d \times d}$ are learnable parameters.

4.5.3 Second-Order Aggregation. To further capture collaborative relationships, the second-order aggregation incorporates information from second-order neighbors. Specifically, prior to the current interaction, user u may have watched several live rooms, which are connected to the current room through the same user, thereby implying similar information. Similarly, different users within a live room are connected through the same room. The same principle applies to interactions between users and streamers. Therefore, the second-order neighbors also play a role in the embedding updating process, which can be formulated as follows:

$$\begin{aligned} \mathbf{e}_{u,t}^{(2)} &= \phi_u \left(\mathbf{e}_{u,t^-}, \mathbf{e}_{u_1,t^-}, \mathbf{e}_{u_2,t^-}, \dots, \mathbf{e}_{u_n,t^-} \right), \quad u_k \in \mathcal{H}_{i,t}, \\ \mathbf{e}_{i,t}^{(2)} &= \phi_i \left(\mathbf{e}_{i,t^-}, \mathbf{e}_{i_1,t^-}, \mathbf{e}_{i_2,t^-}, \dots, \mathbf{e}_{i_m,t^-} \right), \quad i_j \in \mathcal{H}_{u,t}, \end{aligned} \quad (5)$$

where $\mathbf{e}_{u,t}^{(2)}$ and $\mathbf{e}_{i,t}^{(2)}$ denote the embeddings of user u and item i , respectively, which are obtained through second-order aggregation. $\mathcal{H}_{i,t}$ and $\mathcal{H}_{u,t}$ represent the historical neighbor sets of item i and user u at timestamp t . We employ the graph attention aggregation function, whose detailed formulation can be found in Appendix B.

4.5.4 Fusion and Updating. After the aforementioned three updating mechanisms, it is necessary to combine the three types of embeddings to learn the final embeddings at timestamp t . The LiveCI introduced in Section 4.4 will guide the fusion process of the U-R graph, serving as a weight to adjust the impact of collaboration information on embedding learning. The formulation is as follows:

$$\begin{aligned} \mathbf{e}_{u,t} &= F_{u,R} \left(\mathbf{W}_{u,R}^{(0)} \mathbf{e}_{u,t}^{(0)} + (\beta \text{LU}_{u,t}(r) + \alpha) \left(\mathbf{W}_{u,R}^{(1)} \mathbf{e}_{u,t}^{(1)} + \mathbf{W}_{u,R}^{(2)} \mathbf{e}_{u,t}^{(2)} \right) \right), \\ \mathbf{e}_{r,t} &= F_r \left(\mathbf{W}_r^{(0)} \mathbf{e}_{r,t}^{(0)} + (\beta \text{LR}_{r,t}(u) + \alpha) \left(\mathbf{W}_r^{(1)} \mathbf{e}_{r,t}^{(1)} + \mathbf{W}_r^{(2)} \mathbf{e}_{r,t}^{(2)} \right) \right), \end{aligned} \quad (6)$$

where $\mathbf{e}_{u,t}, \mathbf{e}_{r,t} \in \mathbb{R}^d$ are the learned embeddings of user u and room r at timestamp t , respectively. $\mathbf{W}_{u,R}, \mathbf{W}_r \in \mathbb{R}^{d \times d}$ are the learnable parameters of the U-R graph, while $F_{u,R}$ and F_r are the fusion functions. The hyperparameters α, β satisfy $\alpha + \beta = 1$ and are used to adjust the range of LiveCI values according to the scenario. The analysis of the rationale behind the effect of LiveCI can be found in Appendix A.

Notably, the U-S graph, which is based on gifting behavior, is sparse and capable of representing relatively long-term characteristics. Consequently, its fusion aligns with DGCF [19]. The fusion within the U-S graph can be formulated as follows:

$$\begin{aligned} \mathbf{e}_{u,s,t} &= F_{u,S} \left(\mathbf{W}_{u,S}^{(0)} \mathbf{e}_{u,s,t}^{(0)} + \mathbf{W}_{u,S}^{(1)} \mathbf{e}_{u,s,t}^{(1)} + \mathbf{W}_{u,S}^{(2)} \mathbf{e}_{u,s,t}^{(2)} \right), \\ \mathbf{e}_{s,t} &= F_s \left(\mathbf{W}_s^{(0)} \mathbf{e}_{s,t}^{(0)} + \mathbf{W}_s^{(1)} \mathbf{e}_{s,t}^{(1)} + \mathbf{W}_s^{(2)} \mathbf{e}_{s,t}^{(2)} \right), \end{aligned} \quad (7)$$

where $\mathbf{e}_{u,s,t}, \mathbf{e}_{s,t} \in \mathbb{R}^d$ are the learned embeddings of user u and streamer s at timestamp t , respectively. $\mathbf{W}_{u,S}, \mathbf{W}_s \in \mathbb{R}^{d \times d}$ are the learnable parameters of the U-S graph, while $F_{u,S}$ and F_s are the fusion functions. It is worth emphasizing that the user embedding in both the U-R graph and the U-S graph is shared, while the parameters are not shared. When a new interaction I_t occurs, the U-R graph is first updated based on $\mathbf{w}_t \in I_t$. If $g_t \in I_t, g_t = 0$, then the updated $\mathbf{e}_{u,t}$ from the U-R graph becomes the ultimate representation of user u at time t . If $g_t \neq 0$, the U-S graph is further updated based on $\mathbf{e}_{u,t}$, resulting in $\mathbf{e}_{u,s,t}$, which serves as the ultimate representation of user u at time t .

4.6 Prediction and Optimization

In dynamic live room recommendation, our objective is to predict the room that a user is most likely to watch at the next timestamp, based on their historical interaction sequence. Consistent with [17, 19], DCGLive makes recommendations by predicting the future room embedding for the user based on learned representations. To achieve this, the model first projects the user's current embedding to the future timestamp, under the assumption that all embeddings in the dynamic graph evolve smoothly in a continuous latent space. The interval between the current and future timestamps is explicitly incorporated into this projection to capture temporal dynamics. Based on the projected user embedding, DCGLive then predicts the corresponding future room embedding and generates recommendations accordingly. For optimization, each graph is trained independently, with its loss composed of two parts: the Mean Squared Error (MSE) loss and an L2 regularization term. To further improve training efficiency, DCGLive adopts the T-Batch algorithm [17], as commonly used in prior works [19, 32]. The detailed algorithm is provided in Appendix C.

4.6.1 Projection. The projection function for user embedding, which takes into account the time interval Δt_u between the current timestamp t and the next timestamp t^+ , can be formulated as follows:

$$\hat{\mathbf{e}}_{u,t^+} = \text{MLP}_u (\mathbf{e}_{u,t} \odot (\mathbf{1} + \mathbf{w}_t \Delta t_u)), \quad (8)$$

where $\mathbf{w}_t \in \mathbb{R}^d$ represents a learnable parameter vector that transforms the time interval into a d -dimensional embedding space, and $\hat{\mathbf{e}}_{u,t^+}$ denotes the predicted user embedding at the future timestamp t^+ . MLP denotes a Multi-Layer Perceptron (i.e., a feed-forward neural network), and $\odot$ represents the element-wise product.

4.6.2 Prediction. Based on the estimated user embedding $\hat{\mathbf{e}}_{u,t^+}$, our model is capable of predicting the expected room embedding as:

$$\hat{\mathbf{e}}_{r,t^+} = \text{MLP}_r (\mathbf{W}_2 \hat{\mathbf{e}}_{u,t^+} + \mathbf{W}_3 \mathbf{f}_u + \mathbf{W}_4 \mathbf{f}_r), \quad (9)$$

where $\mathbf{W}_2, \mathbf{W}_3$, and $\mathbf{W}_4$ are learnable parameter matrices, and $\hat{\mathbf{e}}_{r,t^+}$ represents the predicted room embedding generated by our model.

4.6.3 Optimization. The loss function for the U-R graph $\mathcal{L}_{UR}$ is formally written as follows:

$$\mathcal{L}_{UR} = \sum_{\{I_t | t \in \mathcal{T}\}} \|\hat{\mathbf{e}}_{r,t^+} - \mathbf{e}_{r,t}\|^2 + \lambda_u \|\mathbf{e}_{u,t} - \mathbf{e}_{u,t^-}\|_2 + \mu_r \|\mathbf{e}_{r,t} - \mathbf{e}_{r,t^-}\|_2, \quad (10)$$

where the first term represents MSE of the prediction, and the latter two terms impose an L2 regularization constraint on the

Table 1: Statistics of the experimental datasets. “U-R Density” and “U-S Density” denote the respective densities of the U-R and U-S graphs.

Datasets	#Users	#Rooms	#Streamers	#Interactions	U-R Density	U-S Density
Comm.	9,667	2,441	850	493,212	2.0901%	0.0036%
KuaiLive	9,574	2,951	749	257,811	0.9125%	0.0051%

updating process. The coefficients λ_u and μ_r control the extent of this smoothing to prevent excessive embedding drift.

Accordingly, in the U-S graph:

$$\hat{\mathbf{e}}_{u,s,t^+} = \text{MLP}_{u,S} (\mathbf{e}_{u,s,t} \odot (\mathbf{1} + \mathbf{w}_t \Delta t_s)), \quad (11)$$

$$\hat{\mathbf{e}}_{s,t^+} = \text{MLP}_s (\mathbf{W}_5 \hat{\mathbf{e}}_{u,s,t^+} + \mathbf{W}_6 \mathbf{f}_u + \mathbf{W}_7 \mathbf{f}_s), \quad (12)$$

$$\mathcal{L}_{US} = \sum_{\{I_t | t \in \mathcal{T}, g_t \neq 0\}} \left(\|\hat{\mathbf{e}}_{s,t^+} - \mathbf{e}_{s,t}\|^2 + \lambda_{u,S} \|\mathbf{e}_{u,s,t} - \mathbf{e}_{u,s,t^-}\|_2 + \mu_s \|\mathbf{e}_{s,t} - \mathbf{e}_{s,t^-}\|_2 \right), \quad (13)$$

where $\hat{\mathbf{e}}_{u,s,t^+}$ denotes the estimated user embedding at the future timestamp t^+ , $\hat{\mathbf{e}}_{s,t^+}$ represents the predicted streamer embedding. $\mathbf{W}_5, \mathbf{W}_6$, and $\mathbf{W}_7$ are learnable parameters. The overall learning algorithm of DCGLive is detailed in Appendix D.

5 Experiments

5.1 Experimental Setups

5.1.1 Datasets. We conduct extensive experiments on both commercial and public datasets (statistics in Table 1):

Commercial dataset. This dataset is constructed from the behavioral logs of 9,667 active users on a popular live streaming platform, covering a three-week period from December 2024 to January 2025. It includes each user's watch duration in live rooms and their gifting behavior with streamers.

KuaiLive dataset¹. KuaiLive is a public dataset for live streaming recommendations, collected from KuaiShou. From the original dataset, we sample and filter to obtain logs containing 9,574 active users for the targeted live room recommendation task.

5.1.2 Evaluation Protocols. Following previous works [19, 32, 40], we employ the interaction-based all-ranking strategy for evaluation. For each interaction, the ground-truth room is ranked by calculating the L_2 distance between the predicted room representation and all rooms. The data are divided chronologically, with 80% for training, 10% for validation, and the remaining 10% for testing. The results of our DCGLive and the baseline models are evaluated using NDCG@{5, 10}, Recall@{5, 10}, and Mean Reciprocal Rank (MRR).

5.1.3 Baselines. To demonstrate the effectiveness of our proposed approach, we compare DCGLive with two categories of competitive baselines: general recommendation methods based on collaborative filtering (CF) and dynamic recommendation methods. Notably, we adapt general methods to align with dynamic room recommendation for interaction-based all-ranking evaluation, as these methods are inherently static. The general methods include BPRMF [27],

¹<https://imgk574.github.io/KuaiLive/>

Table 2: Performance comparison between DCGLive and the baselines on both commercial and public datasets. The best and second-best performance methods are highlighted in bold and underlined fonts, respectively. * indicates p-value < 0.05 from t-test between DCGLive and the best baseline.

Datasets		KuaiLive					Commercial				
Category	Methods	NDCG@5	NDCG@10	Recall@5	Recall@10	MRR	NDCG@5	NDCG@10	Recall@5	Recall@10	MRR
General	BPRMF	0.0009	0.0013	0.0016	0.0028	0.0027	0.0011	0.0019	0.0016	0.0042	0.0020
	NGCF	7.7e-5	0.0002	0.0001	0.0004	0.0018	0.0008	0.0012	0.0013	0.0025	0.0018
	LightGCN	0.0004	0.0006	0.0007	0.0014	0.0020	0.0011	0.0014	0.0019	0.0028	0.0019
	CAGCN	0.0008	0.0010	0.0014	0.0020	0.0024	0.0008	0.0015	0.0015	0.0039	0.0016
Dynamic	DeepCoevolve	0.0559	0.0744	0.0910	0.1485	0.0675	0.0820	0.1017	0.1223	0.1838	0.0921
	JODIE	<u>0.0838</u>	0.1074	<u>0.1292</u>	0.2028	0.0974	0.1650	0.1973	0.2488	0.3493	0.1683
	DGCF	<u>0.0816</u>	<u>0.1118</u>	0.1261	<u>0.2210</u>	<u>0.1081</u>	<u>0.2071</u>	<u>0.2412</u>	<u>0.2926</u>	<u>0.3985</u>	<u>0.2091</u>
	DGEL	0.0017	0.0035	0.0037	0.0092	0.0071	0.0121	0.0189	0.0210	0.0423	0.0212
	NeuFilter	0.0738	0.0970	0.1154	0.1875	0.0891	0.1467	0.1748	0.2247	0.3116	0.1468
Ours	DCGLive	0.0996*	0.1336*	0.1561*	0.2614*	0.1170	0.2219*	0.2593*	0.3304*	0.4463*	0.2149*

NGCF [36], LightGCN [14], and CAGCN [37]. The dynamic methods include DeepCoevolve [6], JODIE [17], DGCF [19], DGEL [32], and NeuFilter [40]. For more details, please refer to Appendix E.1.

5.1.4 Implementation Details. We use the Adam optimizer [16] to train DCGLive. The learning rates for optimizing the parameters of the U-R and U-S graphs are meticulously chosen from $\{1e-3, 5e-4, 1e-4, 5e-4, 1e-5\}$, respectively. The hyperparameter α is selected from the range $\{0.0, 0.1, 0.2, \dots, 1.0\}$. The dimensions of dynamic embedding are selected from $\{16, 32, 64, 128, 256\}$, and to ensure a fair comparison, we set the embedding size of all methods to 128. All experiments are conducted on NVIDIA Tesla V100 32G GPUs. More details about the implementation can be found in Appendix E.2.

5.2 Experimental Results

We compare our proposed DCGLive with the aforementioned baselines on both commercial and public datasets, as shown in Table 2. Overall, DCGLive significantly outperforms all other methods, underscoring the benefits of integrating dynamic room-level collaboration in live streaming recommendations. From the experimental results, we observe that: (1) Dynamic methods significantly outperform static baselines, as the latter fail to capture the rapidly evolving nature of live rooms and user preferences. (2) DGCF emerges as the strongest baseline in most cases, particularly on the commercial dataset where it consistently achieves the second-best performance, underscoring the importance of collaborative relationships in live streaming recommendation. (3) By adeptly modeling tripartite relationships and fine-grained intra-room collaboration, DCGLive outperforms all baselines, achieving improvements of 17.14% and 8.47% on the KuaiLive and Commercial datasets, respectively.

5.3 Ablation Study

We conduct ablation studies by systematically removing each component from DCGLive one at a time. The results, presented in Table 3, demonstrate that each component plays a crucial role in modeling the dynamics of room-level collaborative information. “w/o U-S” denotes the exclusion of the U-S graph. Under this configuration, the model’s performance is comparable to DGCF but inferior to the full DCGLive model. This indicates that integrating the U-S

Table 3: Ablation study of the proposed DCGLive on both commercial and public datasets. “w/o” represents “without”.

Datasets	Settings	NDCG@5	NDCG@10	Recall@5	Recall@10	MRR
Comm.	w/o U-S	0.2068	0.2364	0.2989	0.3903	0.2041
	w/o Init	0.2025	0.2390	0.3033	0.4161	0.1980
	w/o LiveCI	0.2165	0.2536	0.3173	0.4321	0.2122
	DCGLive	0.2219	0.2593	0.3304	0.4463	0.2149
KuaiLive	w/o U-S	0.0922	0.1204	0.1406	0.2282	0.1051
	w/o Init	0.0893	0.1236	0.1396	0.2468	0.1103
	w/o LiveCI	0.0977	0.1305	0.1525	0.2550	0.1158
	DCGLive	0.0996	0.1336	0.1561	0.2614	0.1170

graph offers valuable tripartite collaborative information, which serves as a foundation for subsequent workflows. “w/o Init” signifies the removal of the room representation initialization mechanism. The subsequent performance degradation demonstrates that this mechanism effectively mitigates the cold-start problem for newly created rooms by leveraging both historical room characteristics and the streamer’s current representation. “w/o LiveCI” indicates the exclusion of LiveCI. The decline in performance under this setting confirms that LiveCI enables the model to capture fine-grained collaborative effects among co-watching users, thereby facilitating more effective learning of user and room representations.

5.4 Further Analysis

5.4.1 Analysis on Cold-Start Rooms. To evaluate the performance of DCGLive in cold-start rooms, we conduct an assessment during the testing phase. Specifically, we track all subsequent predictions for rooms that appeared for the first time during this phase. The results are presented in Table 4. DCGLive achieves substantial improvements over both DGCF and the variant “w/o Init” on cold-start rooms across both datasets. These results confirm that our initialization mechanism effectively addresses the cold-start challenge. By leveraging both the streamer’s historical room characteristics and the current state, it enables the generation of meaningful recommendations for newly created rooms, even with limited interaction history. Notably, even the “w/o Init” variant outperforms DGCF, demonstrating that our dual-graph architecture provides benefits beyond the initialization mechanism alone.

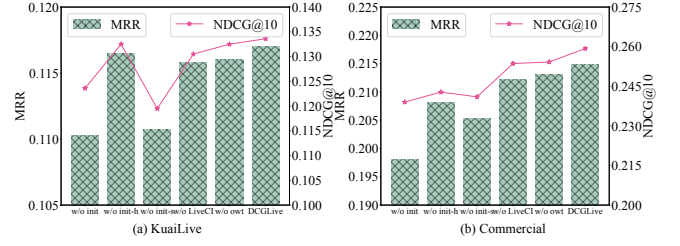
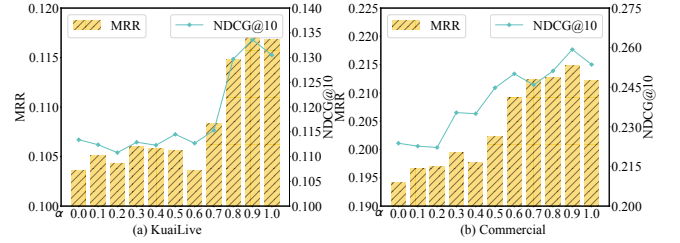
Table 4: Performance on cold-start rooms. N@10 denotes NDCG@10, R@10 denotes Recall@10.

Datasets	KuaiLive			Commercial		
	N@10	R@10	MRR	N@10	R@10	MRR
DGCF	0.0968	0.2109	0.0921	0.2397	0.3858	0.2095
w/o init	0.1165	0.2432	0.1032	0.2487	0.4307	0.2062
DCGLive	0.1368	0.2854	0.1169	0.2671	0.4569	0.2219

5.4.2 Analysis on Variants of Room Representation Initialization Mechanism and LiveCI. To further investigate the effectiveness of DCGLive, we conduct comprehensive experiments on variants of the room representation initialization mechanism and LiveCI. The experimental results are presented in Figure 3. Specifically, “w/o init-h” and “w/o init-s” denote the exclusion of historical room characteristics and the streamer’s latest representation, respectively. The removal of either component results in performance degradation, with the exclusion of the streamer’s latest representation causing a more substantial decline. Notably, even this degraded variant still outperforms the “w/o Init” baseline, which removes the entire representation initialization mechanism. These results demonstrate that the room representation initialization mechanism effectively alleviates the cold-start problem for newly created rooms by jointly incorporating relevance and temporal dynamics across rooms hosted by the same streamer. Moreover, the more pronounced impact of removing the streamer’s latest representation suggests that temporal dynamics play a more critical role than historical patterns for next-room recommendation. Regarding LiveCI, “w/o owt” denotes the removal of the overlapping watching weight. This variant yields performance inferior to the full DCGLive but superior to “w/o LiveCI”, which excludes the entire LiveCI. This observation validates the effectiveness of incorporating overlapping watching weight to better capture intra-room collaborative effect.

5.4.3 Hyper-Parameter Analysis. We conduct experiments to examine the hyper-parameter α in Eq. (6). In our scenarios, we observe that excessively low LiveCI values can adversely affect representation updating. To mitigate this issue, we employ a linear mapping that transforms the LiveCI range to better balance the model’s learning capability with its sensitivity to collaborative effects. Specifically, the hyper-parameter α is used to linearly map the LiveCI values to the interval $[\alpha, 1]$. We evaluate α across the set $\{0.1, 0.2, \dots, 1.0\}$, where $\alpha = 1.0$ corresponds to the baseline without LiveCI incorporation. As shown in Figure 4, optimal performance is achieved when α is set to 0.9, which outperforms the baseline. These findings provide valuable insights into how collaborative effects influence representation updating in DGNNs and guide the selection of appropriate hyper-parameter values for balancing representation learning and collaborative information.

5.4.4 Complexity Analysis. During the training phase, optimization for DCGLive is executed using the T-Batch gradient descent method. The complexity of forming the batches is $O(|I_{tr}|)$, where $|I_{tr}| = |\mathcal{E}_{tr}^{UR}|$ denotes the size of the training dataset. For each interaction involving two nodes in either the U-R or U-S graphs, the complexity of a single updating and prediction session is $O(2N_{\max}d)$, where $N_{\max}$ represents the maximum node degree and d is the embedding dimension. The primary computational cost arises from

**Figure 3: Analysis on variants of room representation initialization mechanism and LiveCI.****Figure 4: Hyper-parameter analysis w.r.t. α .**

the second-order aggregation. Consequently, the worst-case training complexity per epoch is $O(2N_{\max}d(|\mathcal{E}_{tr}^{UR}| + |\mathcal{E}_{tr}^{US}|))$. Compared to DGCF, DCGLive introduces additional computational overhead due to the incorporation of the U-S graph. However, since the U-S graph is considerably sparser than the U-R graph, the number of edges $|\mathcal{E}_{tr}^{US}|$ is significantly smaller than $|\mathcal{E}_{tr}^{UR}|$, i.e., $|\mathcal{E}_{tr}^{US}| \ll |\mathcal{E}_{tr}^{UR}|$. Therefore, the overall model complexity remains acceptable and satisfies the efficiency requirements of our application scenario. Furthermore, the computation of LiveCI can be performed during the data preprocessing phase, thereby reducing the training cost.

6 Conclusion

In this paper, we introduce DCGLive, a dynamic collaboration-aware dual-graph learning approach for live streaming recommendation. This approach effectively leverages the dynamics of room-level collaboration information from two perspectives: *the evolving tripartite relationships among streamers, rooms, and users*, and *the real-time intra-room collaboration information*. Specifically, DCGLive integrates a room initialization mechanism alongside LiveCI to address the room cold-start issue and to capture the fine-grained intra-room collaborative effects. Extensive experimental results demonstrate the effectiveness of DCGLive.

7 Acknowledgments

This work was partially supported by the National Natural Science Foundation of China (No. 62376275, 62472426). Work partially done at Beijing Key Laboratory of Research on Large Models and Intelligent Governance, and Engineering Research Center of Next-Generation Intelligent Search and Recommendation, MOE. Supported by fund for building world-class universities (disciplines) of Renmin University of China. Supported by Kuaishou Technology.

References

- [1] Jiangxia Cao, Shen Wang, Yue Li, Shenghui Wang, Jian Tang, Shiyao Wang, Shuang Yang, Zhaojie Liu, and Guorui Zhou. 2024. Moment&Cross: Next-Generation Real-Time Cross-Domain CTR Prediction for Live-Streaming Recommendation at Kuaishou. *arXiv preprint arXiv:2408.05709* (2024).
- [2] Shiyu Chang, Yang Zhang, Jiliang Tang, Dawei Yin, Yi Chang, Mark A Hasegawa-Johnson, and Thomas S Huang. 2017. Streaming Recommender Systems. In *Proceedings of the 26th International Conference on World Wide Web*. 381–389.
- [3] Edgar Chen, Mark Ally, Eder Santana, and Saad Ali. 2022. Weighing dynamic availability and consumption for Twitch recommendations. (2022).
- [4] Wanyu Chen, Fei Cai, Honghui Chen, and Maarten De Rijke. 2019. A dynamic co-attention network for session-based recommendation. In *Proceedings of the 28th ACM international conference on information and knowledge management*. 1461–1470.
- [5] Xinyue Chen, Si Chen, Xu Wang, and Yun Huang. 2021. "I was afraid, but now I enjoy being a streamer!" Understanding the Challenges and Prospects of Using Live Streaming for Online Education. *Proceedings of the ACM on Human-Computer Interaction* 4, CSCW3 (2021), 1–32.
- [6] Hanjun Dai, Yichen Wang, Rakshit Trivedi, and Le Song. 2016. Deep coevolutionary network: Embedding user and item features for recommendation. *arXiv preprint arXiv:1609.03675* (2016).
- [7] Sunhao Dai, Changle Qu, Sirui Chen, Xiao Zhang, and Jun Xu. 2024. ReCODE: Modeling Repeat Consumption with Neural ODE. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*.
- [8] Jiaxin Deng, Dong Shen, Shiyao Wang, Xiangyu Wu, Fan Yang, Guorui Zhou, and Gaofeng Meng. 2023. ContentCTR: Frame-level Live Streaming Click-Through Rate Prediction with Multimodal Transformer. *arXiv preprint arXiv:2306.14392* (2023).
- [9] Jiaxin Deng, Shiyao Wang, Dong Shen, Liqin Zhao, Fan Yang, Guorui Zhou, and Gaofeng Meng. 2024. A Multimodal Transformer for Live Streaming Highlight Prediction. In *2024 IEEE International Conference on Multimedia and Expo (ICME)*. IEEE, 1–6.
- [10] Jiaxin Deng, Shiyao Wang, Yuchen Wang, Jiansong Qi, Liqin Zhao, Guorui Zhou, and Gaofeng Meng. 2024. MMBee: Live Streaming Gift-Sending Recommendations via Multi-Modal Fusion and Behaviour Expansion. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 4896–4905.
- [11] Ge Gao, Hongyan Liu, and Kang Zhao. 2023. Live streaming recommendations based on dynamic representation learning. *Decision Support Systems* 169 (2023), 113957.
- [12] Will Hamilton, Zhitao Ying, and Jure Leskovec. 2017. Inductive representation learning on large graphs. *Advances in neural information processing systems* 30 (2017).
- [13] Bowei He, Xu He, Yingxue Zhang, Ruiming Tang, and Chen Ma. 2023. Dynamically expandable graph convolution for streaming recommendation. In *Proceedings of the ACM Web Conference 2023*. 1457–1467.
- [14] Xiangnan He, Kuan Deng, Xiang Wang, Yan Li, Yongdong Zhang, and Meng Wang. 2020. Lightgcn: Simplifying and powering graph convolution network for recommendation. In *Proceedings of the 43rd International ACM SIGIR conference on research and development in Information Retrieval*. 639–648.
- [15] Sepp Hochreiter and Jürgen Schmidhuber. 1997. Long Short-Term Memory. *Neural Computation* 9, 8 (1997), 1735–1780. doi:10.1162/neco.1997.9.8.1735
- [16] Diederik P Kingma. 2014. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980* (2014).
- [17] Srijan Kumar, Xikun Zhang, and Jure Leskovec. 2019. Predicting dynamic embedding trajectory in temporal interaction networks. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*. 1269–1278.
- [18] Xiaodong Li, Ruochen Yang, Shuang Wen, Shen Wang, Yueyang Liu, Guoquan Wang, Weisong Hu, Qiang Luo, Jiawei Sheng, Tingwen Liu, et al. 2025. FARM: Frequency-Aware Model for Cross-Domain Live-Streaming Recommendation. *arXiv preprint arXiv:2502.09375* (2025).
- [19] Xiaohan Li, Mengqi Zhang, Shu Wu, Zheng Liu, Liang Wang, and Philip S Yu. 2020. Dynamic graph collaborative filtering. In *2020 IEEE international conference on data mining (ICDM)*. IEEE, 322–331.
- [20] Fengqi Liang, Baigong Zheng, Liqin Zhao, Guorui Zhou, Qian Wang, and Yanan Niu. 2024. Ensure Timeliness and Accuracy: A Novel Sliding Window Data Stream Paradigm for Live Streaming Recommendation. *arXiv preprint arXiv:2402.14399* (2024).
- [21] Yueyang Liu, Jiangxia Cao, Shen Wang, Shuang Wen, Xiang Chen, Xiangyu Wu, Shuang Yang, Zhaojie Liu, Kun Gai, and Guorui Zhou. 2025. LLM-Alignment Live-Streaming Recommendation. *arXiv preprint arXiv:2504.05217* (2025).
- [22] Yucheng Lu, Jiangxia Cao, Xu Kuan, Wei Cheng, Wei Jiang, Jiaming Zhang, Yang Shuang, Liu Zhaojie, and Liyin Hong. 2025. LiveForesighter: Generating Future Information for Live-Streaming Recommendations at Kuaishou. *arXiv preprint arXiv:2502.06557* (2025).
- [23] Aldo Pareja, Giacomo Domeniconi, Jie Chen, Tengfei Ma, Toyotaro Suzumura, Hiroki Kanezashi, Tim Kaler, Tao Schardl, and Charles Leiserson. 2020. Evolvegc: Evolving graph convolutional networks for dynamic graphs. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 34. 5363–5370.
- [24] Changle Qu, Sunhao Dai, Ke Guo, Liqin Zhao, Yanan Niu, Xiao Zhang, and Jun Xu. 2025. KuaLive: A Real-time Interactive Dataset for Live Streaming Recommendation. *arXiv preprint arXiv:2508.05633* (2025).
- [25] Changle Qu, Liqin Zhao, Yanan Niu, Xiao Zhang, and Jun Xu. 2025. Bridging Short Videos and Streamers with Multi-Graph Contrastive Learning for Live Streaming Recommendation. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2059–2069.
- [26] Jérémie Rappaz, Julian McAuley, and Karl Aberer. 2021. Recommendation on live-streaming platforms: Dynamic availability and repeat consumption. In *Proceedings of the 15th ACM Conference on Recommender Systems*. 390–399.
- [27] Steffen Rendle, Christoph Freudenthaler, Zeno Gantner, and Lars Schmidt-Thieme. 2009. BPR: Bayesian personalized ranking from implicit feedback. In *Proceedings of the Twenty-Fifth Conference on Uncertainty in Artificial Intelligence* (Montreal, Quebec, Canada) (UAI '09). AUAI Press, Arlington, Virginia, USA, 452–461.
- [28] Chenglei Shen, Xiao Zhang, Wei Wei, and Jun Xu. 2023. HyperBandit: Contextual bandit with hypernetwork for time-varying user preferences in streaming recommendation. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management (CIKM 2023)*. 2239–2248.
- [29] Liqiang Song, Ye Bi, Mengqi Yao, Zhenyu Wu, Jianming Wang, and Jing Xiao. 2020. Dream: A dynamic relation-aware model for social recommendation. In *Proceedings of the 29th ACM international conference on information & knowledge management*. 2225–2228.
- [30] Weiping Song, Zhiping Xiao, Yifan Wang, Laurent Charlin, Ming Zhang, and Jian Tang. 2019. Session-based social recommendation via dynamic graph attention networks. In *Proceedings of the Twelfth ACM international conference on web search and data mining*. 555–563.
- [31] Haoran Tang, Shiqing Wu, Xueyao Sun, Jun Zeng, Guandong Xu, and Qing Li. 2025. TCGC: Temporal collaboration-aware graph co-evolution learning for dynamic recommendation. *ACM Transactions on Information Systems* 43, 1 (2025), 1–27.
- [32] Haoran Tang, Shiqing Wu, Guandong Xu, and Qing Li. 2023. Dynamic graph evolution learning for recommendation. In *Proceedings of the 46th international acm sigir conference on research and development in information retrieval*. 1589–1598.
- [33] Md Mehrab Tanjim, Hammad A Ayubi, and Garrison W Cottrell. 2020. DynamicRec: a dynamic convolutional network for next item recommendation. In *Proceedings of the 29th ACM international conference on information & knowledge management*. 2237–2240.
- [34] Junshan Wang, Guojie Song, Yi Wu, and Liang Wang. 2020. Streaming graph neural networks via continual learning. In *Proceedings of the 29th ACM international conference on information & knowledge management*. 1515–1524.
- [35] Lu Wang, Wei Zhang, Xiaofeng He, and Hongyuan Zha. 2018. Supervised reinforcement learning with recurrent neural network for dynamic treatment recommendation. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*. 2447–2456.
- [36] Xiang Wang, Xiangnan He, Meng Wang, Fuli Feng, and Tat-Seng Chua. 2019. Neural graph collaborative filtering. In *Proceedings of the 42nd international ACM SIGIR conference on Research and development in Information Retrieval*. 165–174.
- [37] Yu Wang, Yuying Zhao, Yi Zhang, and Tyler Derr. 2023. Collaboration-aware graph convolutional network for recommender systems. In *Proceedings of the ACM web conference 2023*. 91–101.
- [38] Zhihao Wen and Yuan Fang. 2022. Trend: Temporal event and node dynamics for graph representation learning. In *Proceedings of the ACM web conference 2022*. 1159–1169.
- [39] Jiafeng Xia, Dongsheng Li, Hansu Gu, Jiahao Liu, Tun Lu, and Ning Gu. 2022. FIRE: Fast incremental recommendation with graph signal processing. In *Proceedings of the ACM Web Conference 2022*. 2360–2369.
- [40] Jiafeng Xia, Dongsheng Li, Hansu Gu, Tun Lu, Peng Zhang, Li Shang, and Ning Gu. 2024. Neural Kalman filtering for robust temporal recommendation. In *Proceedings of the 17th ACM International Conference on Web Search and Data Mining*. 836–845.
- [41] Da Xu, Chuanwei Ruan, Evren Korpeoglu, Sushant Kumar, and Kannan Achan. 2020. Inductive representation learning on temporal graphs. *arXiv preprint arXiv:2002.07962* (2020).
- [42] Jiaxuan You, Yichen Wang, Aditya Pal, Pong Eksombatchai, Chuck Rosenberg, and Jure Leskovec. 2019. Hierarchical temporal convolutional networks for dynamic recommender systems. In *The world wide web conference*. 2236–2246.
- [43] Sanshi Yu, Zhuoxuan Jiang, Dong-Dong Chen, Shanshan Feng, Dongsheng Li, Qi Liu, and Jinfeng Yi. 2021. Leveraging tripartite interaction information from live stream e-commerce for improving product recommendation. In *Proceedings of the 27th ACM SIGKDD conference on knowledge discovery & data mining*. 3886–3894.
- [44] Mengqi Zhang, Shu Wu, Xueli Yu, Qiang Liu, and Liang Wang. 2022. Dynamic graph neural networks for sequential recommendation. *IEEE Transactions on Knowledge and Data Engineering* 35, 5 (2022), 4741–4753.
- [45] Xiao Zhang, Sunhao Dai, Jun Xu, Zhenhua Dong, Quanyu Dai, and Ji-Rong Wen. 2022. Counteracting user attention bias in music streaming recommendation

- via reward modification. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD 2022)*. 2504–2514.
- [46] Xiao Zhang, Haonan Jia, Hanjing Su, Wenhao Wang, Jun Xu, and Ji-Rong Wen. 2021. Counterfactual reward modification for streaming recommendation with delayed feedback. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR 2021)*. 41–50.
- [47] Yang Zhang, Fuli Feng, Chenxu Wang, Xiangnan He, Meng Wang, Yan Li, and Yongdong Zhang. 2020. How to retrain recommender system? A sequential meta-learning method. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*. 1479–1488.
- [48] Zeyang Zhang, Xin Wang, Haibo Chen, Haoyang Li, and Wenwu Zhu. 2024. Disentangled Dynamic Graph Attention Network for Out-of-Distribution Sequential Recommendation. *ACM Transactions on Information Systems* 43, 1 (2024), 1–42.
- [49] Haiyuan Zhao, Changshuo Zhang, Yang Wang, Hao Wang, Zhen Ouyang, Bin Yuan, Qinglei Wang, and Zuo Tao Liu. 2025. Towards Unbiased and Real-Time Staytime Prediction for Live Streaming Recommendation. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management*. 6293–6300.
- [50] Ziwei Zhao, Xi Zhu, Tong Xu, Aakas Lizhiyu, Yu Yu, Xueying Li, Zikai Yin, and En-hong Chen. 2023. Time-interval aware share recommendation via bi-directional continuous time dynamic graphs. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 822–831.
- [51] Xiaolin Zheng, Jiajie Su, Weiming Liu, and Chaochao Chen. 2022. DDGHM: Dual dynamic graph with hybrid metric training for cross-domain sequential recommendation. In *Proceedings of the 30th ACM international conference on multimedia*. 471–481.
- [52] Yanping Zheng, Lu Yi, and Zhewei Wei. 2025. A survey of dynamic graph neural networks. *Frontiers of Computer Science* 19, 6 (2025), 196323.

A Analysis on collaborative effect

In this section, we provide a detailed analysis of the effects of intra-room collaboration and explore how LiveCI can facilitate the updating process for both users and rooms. This facilitation, in turn, benefits the performance of next-room prediction tasks.

The vanilla updating process of general DGNNs of the U-R graph, excluding LiveCI, can be computed as follows:

$$\begin{aligned} \mathbf{e}_{u,t} &= F_{u,R} \left(\mathbf{W}_{u,R}^{(0)} \mathbf{e}_{u,t}^{(0)} + \mathbf{W}_{u,R}^{(1)} \mathbf{e}_{u,t}^{(1)} + \mathbf{W}_{u,R}^{(2)} \mathbf{e}_{u,t}^{(2)} \right), \\ \mathbf{e}_{r,t} &= F_r \left(\mathbf{W}_r^{(0)} \mathbf{e}_{r,t}^{(0)} + \mathbf{W}_r^{(1)} \mathbf{e}_{r,t}^{(1)} + \mathbf{W}_r^{(2)} \mathbf{e}_{r,t}^{(2)} \right), \end{aligned} \quad (14)$$

here, in each formula, the latter two terms incorporate collaboration information, as both first-order and second-order considerations take into account the node's neighbors. For simplicity, we denote them as $N(u, t)$ and $N(r, t)$, respectively. Thus, Eq. (14) can be rewritten as:

$$\begin{aligned} \mathbf{e}_{u,t} &= F_{u,R} \left(\mathbf{W}_{u,R}^{(0)} \mathbf{e}_{u,t}^{(0)} + N(u, t) \right), \\ \mathbf{e}_{r,t} &= F_r \left(\mathbf{W}_r^{(0)} \mathbf{e}_{r,t}^{(0)} + N(r, t) \right). \end{aligned} \quad (15)$$

Following Tang et al. [31], Wang et al. [37], we use the standard item score function $y_{u,r,t} = \mathbf{e}_{u,t} \cdot \mathbf{e}_{r,t}$ as an example to better illustrate the collaborative effects arising from interactions between users and rooms. Omitting non-linear activation,

$$\begin{aligned} y_{u,r,t} &= \left(\mathbf{W}_{u,R}^{(0)} \mathbf{e}_{u,t}^{(0)} + N(u, t) \right) \cdot \left(\mathbf{W}_r^{(0)} \mathbf{e}_{r,t}^{(0)} + N(r, t) \right) \\ &= \mathbf{W}_{u,R}^{(0)} \mathbf{e}_{u,t}^{(0)} \cdot \mathbf{W}_r^{(0)} \mathbf{e}_{r,t}^{(0)} + N(u, t) \cdot \mathbf{W}_r^{(0)} \mathbf{e}_{r,t}^{(0)} \\ &\quad + \mathbf{W}_{u,R}^{(0)} \mathbf{e}_{u,t}^{(0)} \cdot N(r, t) + N(u, t) \cdot N(r, t). \end{aligned} \quad (16)$$

Focusing on collaboration-aware parts, we can find that: (1) Each historical neighbor, calculated using both first-order and second-order updating, of the current interaction participants (both user

and room) influences the learning process. (2) The collaborative effect is intersectional; in other words, the collaborative informative gain of the current interaction is determined by the extent to which the participants' neighbors match with each other. (3) Furthermore, in live streaming scenarios, as illustrated in Figure 1 (c), the fine-grained temporal information and the watch duration of users within the same room can also reflect intra-room collaboration.

Therefore, we design LiveCI as shown in Eqs. (1), (2), which takes into account both the temporal factor and the intersectional collaborative effect. Additionally, it incorporates the overlapping watch duration factor for $\text{LR}_{r,t}(u)$. The LiveCI then guides the representation learning process of the U-R graph through first-order and second-order updating, as described in Eq. (6), since only these two orders involve collaborative objects. While the U-S graph is relatively sparse, as shown in Table 1, and the gifting behaviors represent users' long-term interests, we do not introduce a similar mechanism to the U-S graph.

B Graph Attention Aggregation Function

In Section 4.5.3, we utilize the graph attention aggregation function for second-order updating. The formulation is detailed as follows. It is worth noting that alternative aggregator functions, such as the Mean aggregator [12] and LSTM aggregator [15], can also be employed during this phase. However, the selection of aggregation functions is not the primary focus of our work.

$$\begin{aligned} \mathbf{e}_{u,t}^{(2)} &= \sum_{u_k \in \mathcal{H}_{i,t}} a_{uk} \mathbf{e}_{u_k,t^-}, \\ a_{uk} &= \frac{\exp(\text{LeakyRelu}(\mathbf{W}_w[\mathbf{e}_{u,t^-} \parallel \mathbf{e}_{u_k,t^-}]))}{\sum_{u_k \in \mathcal{H}_{i,t}} \exp(\text{LeakyRelu}(\mathbf{W}_w[\mathbf{e}_{u,t^-} \parallel \mathbf{e}_{u_k,t^-}]))}, \\ \mathbf{e}_{i,t}^{(2)} &= \sum_{i_j \in \mathcal{H}_{u,t}} a_{ij} \mathbf{e}_{i_j,t^-}, \\ a_{ij} &= \frac{\exp(\text{LeakyRelu}(\mathbf{W}_w[\mathbf{e}_{i,t^-} \parallel \mathbf{e}_{i_j,t^-}]))}{\sum_{i_j \in \mathcal{H}_{u,t}} \exp(\text{LeakyRelu}(\mathbf{W}_w[\mathbf{e}_{i,t^-} \parallel \mathbf{e}_{i_j,t^-}]))}, \end{aligned} \quad (17)$$

where $\mathbf{W}_w \in \mathbb{R}^{2d}$ is a weight matrix, and $\parallel$ denotes the concatenation operation.

C T-Batch Algorithm

We employ the T-Batch algorithm, designed to parallelize training, as introduced by Kumar et al. [17], to train our DCGLive model. The T-Batch algorithm adheres to three key criteria: (1) interactions within each batch are processed simultaneously; (2) the batching process preserves the original sequential order; (3) each batch is free of duplicate nodes to prevent conflicts during node updates. To adapt the T-Batch algorithm for the targeted next-room prediction task, we make several modifications to the original algorithm. Specifically, we designate the item used to determine the batch index as the streamer, since a user cannot simultaneously watch multiple live rooms. The detailed algorithm is presented in Algorithm 1.

D Learning Algorithm of DCGLive

The learning algorithm of DCGLive is presented in Algorithm 2.

```

1 for  $u \in \mathcal{U}$  do
2    $\lfloor \text{last}_U[u] \leftarrow 0;$ 
3 for  $s \in \mathcal{S}$  do
4    $\lfloor \text{last}_S[s] \leftarrow 0;$ 
5 for  $k \in [1, |I|]$  do
6    $\lfloor B_k \leftarrow \emptyset;$ 
7  $C \leftarrow 0;$  // Non-empty batch counter
8 // Batch Assignment Phase
9 for  $t \leftarrow 1$  to  $|I|$  do
10    $(u_t, r_t, s_t, d_t, g_t, t) \leftarrow I_t;$ 
11    $\text{idx} \leftarrow \max(\text{last}_U[u_t], \text{last}_S[s_t]) + 1;$  // Determine
    batch index
12    $B_{\text{idx}} \leftarrow B_{\text{idx}} \cup \{I_t\};$  // Insert interaction into
    batch
13    $\text{last}_U[u_t] \leftarrow \text{idx};$  // Update user's latest batch
14    $\text{last}_S[s_t] \leftarrow \text{idx};$  // Update streamer's latest
    batch
15    $C \leftarrow \max(C, \text{idx});$  // Update batch counter
16 return  $\{B_1, B_2, \dots, B_C\};$ 

```

- **CAGCN**. Similarity metrics: Jaccard Similarity (to maintain consistency with our DCGLive); Propagation layers: $L = 3$; Learning rate: $1e-3$ on commercial dataset, $1e-7$ on KuaiLive dataset.
- **DeepCoevolve**. Learning rate: $1e-3$; Negative rooms: 100; Negative users: 100.
- **JODIE**². Learning rate: $1e-3$; State change: False.
- **DGCF**³. Learning rate: $1e-3$; State change: False; Time span number: 500; Aggregate function: attention (to maintain consistency with our DCGLive).
- **DGEL**⁴. Learning rate: $1e-5$ on commercial dataset, $1e-4$ on KuaiLive dataset; Time span number: 500; Sample length: 100.

- **NeuFilter**⁵. Learning rate: $1e-3$; Propagation layers: $L = 3$; State change: False.

- **DCGLive**: Learning rate: $5e-4$ for optimizing both the U-R and U-S graphs on commercial dataset, $1e-3$ for optimizing the U-R graph and $5e-4$ for optimizing the U-S graph on KuaiLive dataset; Max neighbors for computing LiveCI: 50 on commercial dataset and 30 on KuaiLive dataset; $\alpha = 0.9$; Time span number: 500.

²<https://github.com/claws-lab/jodie>

³<https://github.com/CRIPAC-DIG/DGCF>

⁴<https://github.com/henrictang/DGEL>

⁵<https://github.com/Yaveng/NeuFilter>